BSDA2001P - Introduction to Deep Learning and Generative AI Project

Smart MCQ Solver Challenge

Name: Chittari Sri Siva Siddhartha Varma
Roll: 24f1002705

1. Abstract/Executive Summary

This report covers a deep learning pipeline built for the Kaggle “Smart MCQ Solver Challenge”, a five-option multiple-choice QA task spanning philosophy, physics, and astronomy, scored with Mean Average Precision at 3 (MAP@3). The project progressed from classical baselines (TF-IDF, Word2Vec), through zero-shot transformer methods (Sentence-BERT, BART-MNLI) and a RAG pipeline, to fine-tuned transformer and RNN models. The best approach was DeBERTa-v3-base fine-tuned as a pairwise binary relevance classifier, trained with a two-stage learning-rate schedule and combined into a 5-fold OOF ensemble, reaching a Kaggle MAP@3 of 0.75436, more than double the 0.35250 fixed-guess baseline.

2. Introduction

2.1 - Problem Statement

The competition provides 2,000 training questions (prompt, options A–E, answer) and 500 test questions with the answer withheld. Distractor options are typically full explanatory sentences rather than short factoids, making this closer to reading comprehension than fact lookup. Submissions are scored with MAP@3.

2.2 - Project Objective

The goals were to (1) maximize leaderboard MAP@3 by iterating through classical, zero-shot, and fine-tuned methods, and (2) build practical skill with Hugging Face/PyTorch, tokenization, training loops, W&B tracking, k-fold CV, ensembling alongside GenAI techniques like zero-shot classification and RAG.

2.3 - Report Structure

Section 3 covers the dataset and preprocessing; Section 4 the tokenization strategy; Section 5 the three required model families; Section 6 the full comparative results; Section 7 conclusions and future work;

3. Dataset + Preprocessing

3.1 Dataset Description

Provided by Kaggle (“smart-mcq-solver-challenge”) via kagglehub: train.csv (2,000 rows: id, prompt, A–E, answer), test.csv (500 rows, answer withheld), and sample_submission.csv (fixed "A B C" guess, scoring 0.35250 MAP@3, the project baseline). No missing values in either split.

3.2 EDA

Class balance is mildly skewed toward B/C, which is why the fixed “A B C” guess already beats random chance. Prompts average 18.1 words; options average 26.3 words (up to 118), options are often longer than the question, so the task needs semantic discrimination, not keyword matching. Cleaned prompts contain 859 unique tokens across all 2,000 questions.

3.3 Preprocessing

A clean_text() utility lower-cases and strips punctuation (used for TF-IDF/Word2Vec/BiLSTM; transformers tokenize raw text directly). Stratified splits/folds (on answer, seed 42) preserve class balance throughout. The key preprocessing step was reframing the 5-way question as pairwise binary classification: expand_dataset() turns each question into five (prompt, option, label) rows, 10,000 training pairs total, consumed by both the transformer and BiLSTM models; the five option scores are re-ranked at inference to produce the top-3 answer.

3.4 Data Augmentation

No synthetic augmentation was used; the pairwise expansion above and 5-fold CV were the main ways training signal was stretched from the 2,000 base questions.

4. Tokenization Strategy

4.1 Tokenizer & Justification

4.2 Implementation

- AutoTokenizer.from_pretrained("microsoft/deberta-v3-base") called as tokenizer(prompt, option, truncation=True, max_length=256), standard sentence-pair packing.
- DataCollatorWithPadding pads dynamically per batch rather than to a fixed length.
- A custom MCQDataset expands each question into 5 pairs; DataLoader uses batch_size=16 with pinned memory for the Kaggle T4 GPU.

4.3 - Experimental Comparison

Two alternatives were tried: (1) a from-scratch whitespace vocabulary (build_vocab(), padded to 128 tokens) for the BiLSTM, far cheaper but no pretrained knowledge; (2) RoBERTa-base's BPE tokenizer under an identical 3-epoch schedule, which scored lower than DeBERTa-v3 (0.49958 vs. 0.62177 MAP@3), so DeBERTa-v3 was used for all further runs.

5. Modeling & Experimentation

Four non-trained baselines were tried first to set a performance floor: TF-IDF and Word2Vec cosine similarity, Sentence-BERT cosine similarity, BART-large-MNLI zero-shot classification, and a RAG pipeline (sBERT + FAISS retrieval, cross-encoder rerank, zero-shot scoring). Full scores are in Section 6; none involved gradient-based training on this data.

5.1 Model 1: Scratch Model - BiLSTM

The custom-architecture and RNN-based requirements are both met by the same model: a from-scratch bidirectional LSTM, deliberately hand-built (rather than a second unrelated network) to serve as a lightweight, fully custom sequence baseline against the transformer.

- A trainable nn.Embedding (vocab × 300, no pretrained vectors) feeds a single-layer bidirectional LSTM (hidden_dim=128/direction); concatenated final states (256-dim) pass through Linear(256→128)→ReLU→Dropout(0.3)→Linear(128→1), trained with BCEWithLogitsLoss.
- Bidirectionality helps with the longer, sentence-level options (avg. 26 words) versus a uni-directional RNN.
- Input is the concatenated (prompt, option) text encoded against a from-scratch vocabulary, padded to 128 tokens — much smaller than the transformer pipeline by design.
- Each of the 5 options is scored independently and the top 3 are returned, same as the transformer model.

Trained 5 epochs (Adam, lr=1e-3) on the same 80/20 stratified split. Per-epoch accuracy was tracked, but the notebook's cell outputs were cleared before saving, so no final MAP@3/accuracy number survives for this run — no score is fabricated here. Qualitatively, a from-scratch RNN with no pretraining on 2,000 questions is expected to trail the fine-tuned transformer results below.

5.2 - Model 2: Fine-Tuned Transformer - DeBERTa-v3-base (Single)

- Disentangled attention represents content and relative position separately, helping the model parse the long, syntactically complex options in this dataset.
- The Enhanced Mask Decoder adds absolute position only at the final layer, improving pretraining efficiency.

Fine-tuned end-to-end (no frozen layers) as a binary sequence-pair classifier with AdamW (weight_decay=0.01) and linear warm-up/decay. Trained with two-stage runs continuing 8 epochs (2e-5) with 8 or 5 further epochs at 5e-6. Best checkpoints were chosen by lowest validation loss (W&B) and saved as artifacts. No quantization was applied: models trained/served in float32 on a single Kaggle T4.

5.3 - Model 3: 5-fold OOF Ensembled DeBERTa

The same above model was implemented splitting the data into 5 folds for out of fold training, and then ensembled the 5 models at the end, to average the logits and rank the top 3. This was the most successful model with kaggle score at 0.7546

6. Performance and Comparative Analysis

Model	Validation Loss	Prediction performance (MAP@3)
Baseline (sample_submission.csv)	-	0.3525
TF-IDF	0.29617	0.32294
Word2vec	0.34792	0.30423
sBERT	0.42308	0.38653
BART-MNLI	0.56583	0.49985
DaBERTa (3 epochs)	-	0.62177
RoBERTa (3 epochs)	-	0.49958
DaBERTa (8 epochs at 2e-5)	-	0.74064
DaBERTa (16 epochs at 2e-5)	-	0.71945
DaBERTa (8(2e-5) + 8(5e-6))	-	0.73316
DaBERTa (8(2e-5) + 5(5e-6))	-	0.74397
RAG model (MiniLM-v6)	-	0.68536
DaBERTa (8(2e-5) + 5(5e-6)) 5-fold (without the discarded fold)	-	0.75436
DaBERTa (8(5e-6)) 5-fold	-	0.73815
BiLSTM	-	0.41230

TF-IDF/Word2Vec scored below baseline, lexical similarity carries little signal when distractors share vocabulary with the answer. sBERT and BART-MNLI zero-shot were the first to beat baseline, showing semantic framing matters. RAG added significant engineering for a middling 0.68536, worse than fine-tuning, and was dropped. Fine-tuning DeBERTa gave the largest single jump (0.62 to 0.74 from 3 to 8 epochs); 16 epochs overfit (0.71945), while a two-stage warm restart was the best use of extra compute (0.74397). The 5-fold ensemble added a further, smaller gain at 5 times training cost (0.75436).

7. Conclusion & Future Work

7.1 - Key Learnings

- Converting the 5-option MCQ problem into five binary classification tasks significantly improved model performance.
- Fine-tuned transformer models performed much better than TF-IDF and Word2Vec approaches.
- Two-stage fine-tuning improved performance and reduced overfitting compared to training for more epochs.
- Using a 5-fold ensemble provided a small but consistent improvement over a single model.

7.2 - Challenges Faced

- Some training runs failed to learn properly during the initial epochs, requiring multiple experiments with learning rates and training schedules.
- Finding the right balance between training longer and avoiding overfitting required extensive experimentation.
- The RAG pipeline was more complex to build and achieved lower performance than the fine-tuned transformer model.

7.3 - Future Improvements

- Experiment with larger transformer models such as DeBERTa-v3-Large.
- Improve the BiLSTM model using pretrained word embeddings and evaluate it using MAP@3.
- Explore weighted model ensembling based on validation performance.
- Perform automated hyperparameter tuning using tools like W&B Sweeps.
- Improve the RAG system by using the fine-tuned DeBERTa model for document reranking instead of the zero-shot classifier.

8. References

- He et al. (2021). DeBERTa: Decoding-enhanced BERT with Disentangled Attention. ICLR 2021, arXiv:2006.03654; DeBERTaV3, arXiv:2111.09543. Liu et al. (2019).
- RoBERTa: A Robustly Optimized BERT Pretraining Approach. arXiv:1907.11692. Lewis et al. (2019).
- BART. arXiv:1910.13461 (via facebook/bart-large-mnli). Reimers & Gurevych (2019).
- Sentence-BERT. EMNLP 2019 (via sentence-transformers/all-MiniLM-L6-v2). Mikolov et al. (2013).
- Efficient Estimation of Word Representations in Vector Space. arXiv:1301.3781 (Word2Vec/gensim). Johnson, Douze & Jégou (2019).
- Billion-scale similarity search with GPUs (FAISS). HuggingFaceTransformers/Datasets/Accelerate/PEFT, PyTorch, scikit-learn, gensim, Weights & Biases, kagglehub. Repository: DL-24f1002705-notebook-t22026.